



OASIS Security Report

Authentication Issues Security Analysis

Date: 2026-06-11 09:40:27 — **Model:** mistral:latest

Vulnerability: Authentication Issues

Project: clean_contracts

Analysis root: ../clean

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **3** file(s) for Authentication Issues.

[Return to table of contents](#)

Statistics

Files analyzed: 3 **Total findings:** 3

Critical		0
High		0

Medium**3****Low****0**[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/clean/modifier_reentrancy_fixed.sol	0.590
/work/project/dataset/clean/mycontract_fixed.sol	0.578
/work/project/dataset/clean/simple_dao_fixed.sol	0.577

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.590

Source lines: 1-42

► Finding 1.1: Authentication bypass vulnerability (Medium)

Vulnerable code

```
function airDrop() supportsToken hasNoBalance public{
    tokenBalance[msg.sender] += 20;
}
```

Explanation

The `hasNoBalance` modifier is checked before the `supportsToken` function, allowing any contract with a non-zero balance to access the `airDrop` function.

Impact

An attacker can gain tokens even if they do not have the required initial zero balance.

Entry point

airDrop() function

Execution path

```

'''
+-----+
|  Contract  |
+----|Contract|---
      |      |
      |  Call  |
      |      |
+--|airDrop()|
      |      |
      |  here  |
      |      |
+-----+---->+
      |  Bank  |
+-----+'''
```

Parameters

none

Exploitation steps

- Call the `airDrop()` function with a contract that has a non-zero balance

Conditions

The attacker must have control over a contract that initially has a non-zero balance

Remediation

Ensure the ``hasNoBalance`` check is made before the ``supportsToken`` check to prevent authentication bypass.

Secure example

```
function airDrop() public hasNoBalance supportsToken {  
    tokenBalance[msg.sender] += 20;  
}
```

File 2: /work/project/dataset/clean/mycontract_fixed.sol

Similarity score: 0.578

Source lines: 1-31

► Finding 2.1: Authentication Issue found (Medium)

Vulnerable code

```
function sendTo(address receiver, uint amount) public {  
    require(msg.sender == owner);  
    receiver.transfer(amount);  
}
```

Explanation

The function 'sendTo' uses the sender of the message (msg.sender) to check if the sender is the contract owner before performing the action, making it vulnerable to an attacker who has already authenticated.

Impact

An attacker can transfer funds if they have previously interacted with the contract and gained access to its state

Entry point

sendTo function

Execution path

```
A -> MyContract.sendTo(B, X)  
B -> requires (msg.sender == owner)
```

Parameters

receiver, amount

Exploitation steps

- An attacker interacts with the contract to authenticate
- The attacker calls sendTo function passing their address as 'receiver' and an arbitrary amount

Example payloads

- `sendTo(0xAttacker, 1 ether)`

Conditions

An attacker must have previously interacted with the contract

Remediation

Use the owner variable to check authorization before sending funds and ensure only the contract owner can call sendTo

Secure example

```
function sendTo(address receiver, uint amount) public {  
    require(owner == msg.sender);  
    require(receiver.balance > amount);  
    receiver.transfer(amount);  
}
```

File 3: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.577

Source lines: 1-34

► Finding 3.1: Authentication issue (Medium)

Vulnerable code

```
credit[msg.sender]>= amount
```

Explanation

Weakness in authentication mechanisms, as the contract does not check if the sender is actually authorized to withdraw funds.

Impact

Unauthorized access or privilege escalation

Entry point

withdraw function

Execution path

```
'''  
init -> withdraw  
'''
```

Parameters

amount

Exploitation steps

- Send a transaction to the contract with an invalid 'to' address.
- Call the 'withdraw' function with a large enough 'amount'
- Drain funds from the contract

Example payloads

- 0x...
- 100

Remediation

Implement access control checks before allowing withdrawals, such as requiring the sender to be the owner of the account.

Secure example

```
function withdraw(uint amount) public onlyOwner() {  
    require(msg.sender == owner, 'Caller is not the owner.');
```

```
    if (credit[msg.sender]>= amount) {  
        credit[msg.sender]-=amount;  
        require(msg.sender.call.value(amount)());  
    }  
}
```

```
}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)